

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import models, layers
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score
)

print("TensorFlow version:", tf.__version__)
```

TensorFlow version: 2.20.0

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data(

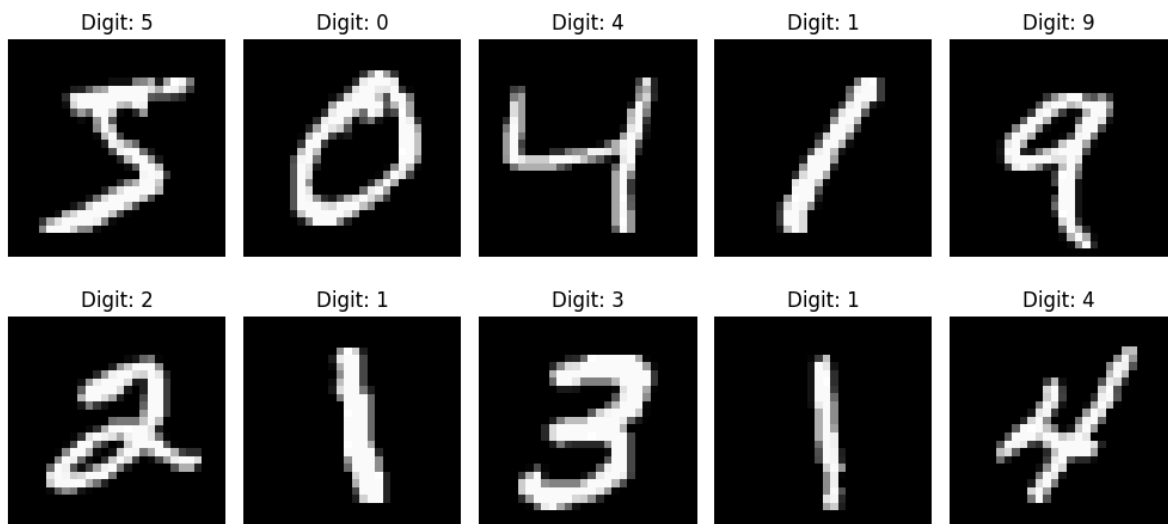
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-d11490434/11490434> ————— 1s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title("Digit: " + str(y_train[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())
```

```
Minimum pixel value: 0.0
Maximum pixel value: 1.0
```

```
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

print("New training shape:", x_train.shape)
print("New testing shape:", x_test.shape)
```

```
New training shape: (60000, 28, 28, 1)
New testing shape: (10000, 28, 28, 1)
```

```
x_val = x_train[-10000:]
y_val = y_train[-10000:]

x_train_new = x_train[:-10000]
y_train_new = y_train[:-10000]

print("Training:", x_train_new.shape)
print("Validation:", x_val.shape)
print("Testing:", x_test.shape)
```

```
Training: (50000, 28, 28, 1)
Validation: (10000, 28, 28, 1)
Testing: (10000, 28, 28, 1)
```

```
model = models.Sequential([

    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
```

```

    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(
        128,
        activation="relu"
    ),

    layers.Dense(
        10,
        activation="softmax"
    )
])

model.summary()

```

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/bas
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

```

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

```

```
print("Model compiled successfully!")
```

Model compiled successfully!

```
history = model.fit(
    x_train_new,
    y_train_new,
    epochs=5,
    batch_size=64,
    validation_data=(x_val, y_val)
)
```

Epoch 1/5

782/782 ————— **25s** 30ms/step - accuracy: 0.9484 - loss: 0.17

Epoch 2/5

782/782 ————— **42s** 31ms/step - accuracy: 0.9852 - loss: 0.04

Epoch 3/5

782/782 ————— **40s** 30ms/step - accuracy: 0.9896 - loss: 0.03

Epoch 4/5

782/782 ————— **42s** 31ms/step - accuracy: 0.9921 - loss: 0.02

Epoch 5/5

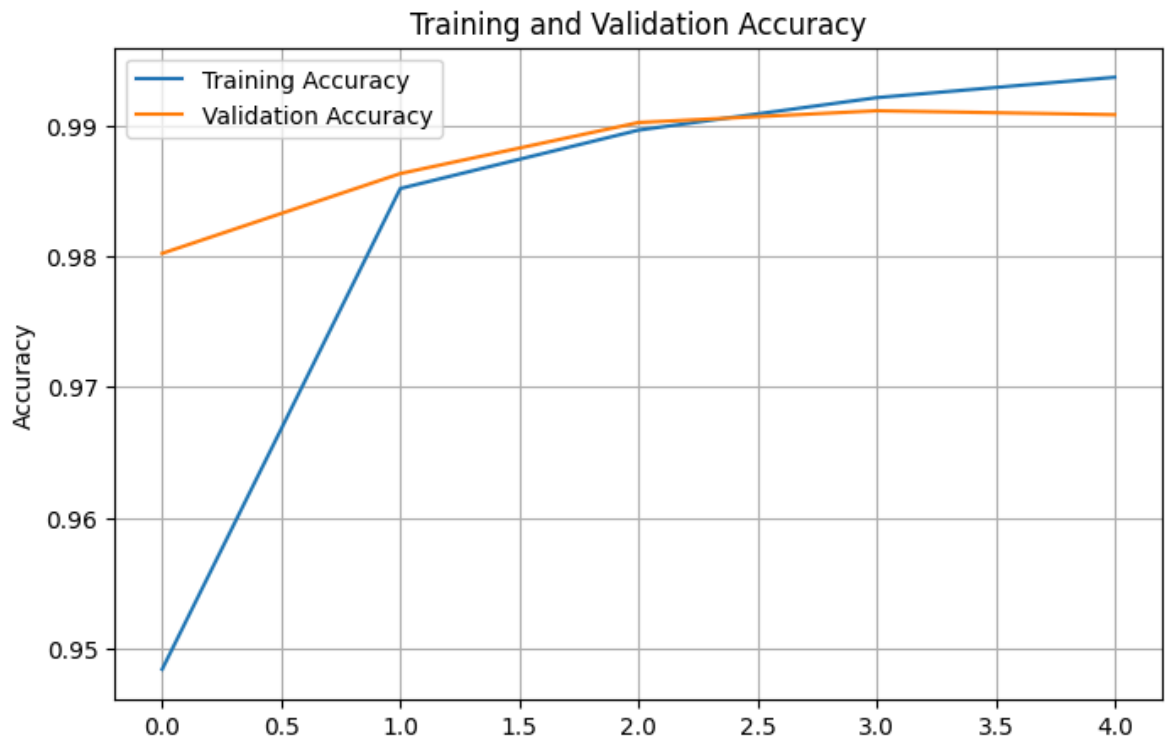
782/782 ————— **41s** 31ms/step - accuracy: 0.9937 - loss: 0.01

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accura

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid()

plt.show()
```

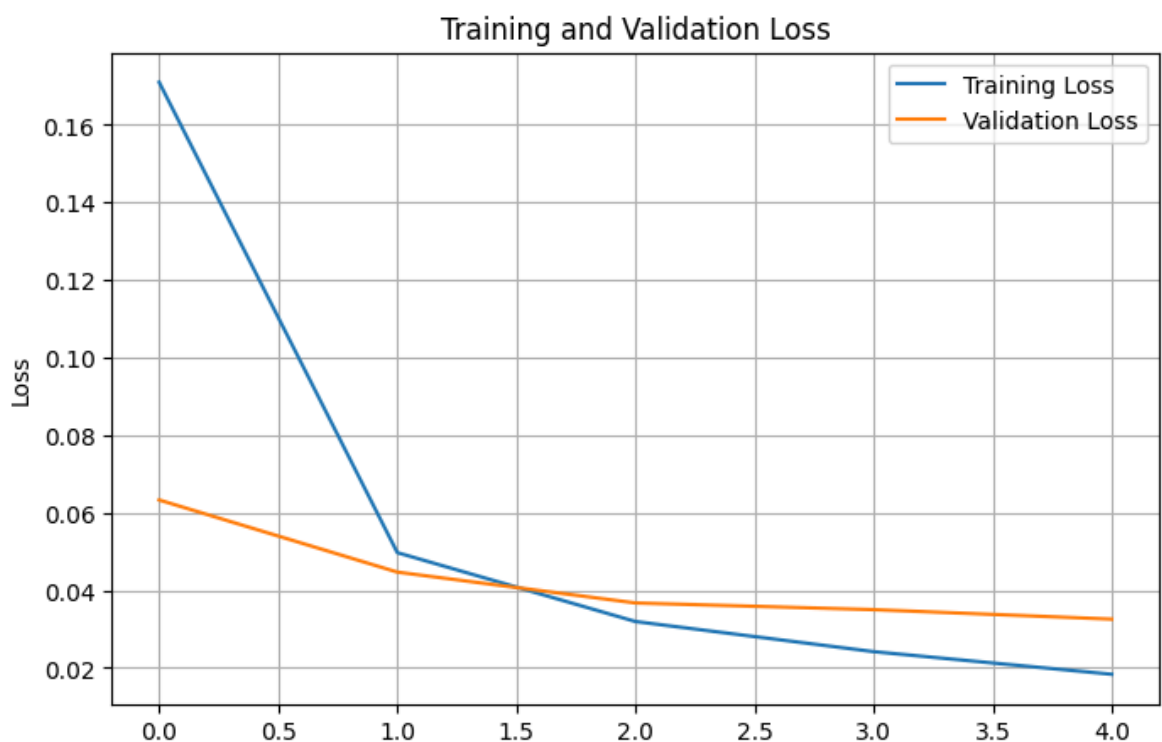


```
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid()

plt.show()
```



```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=1
)
```

```
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

313/313 ————— 2s 5ms/step - accuracy: 0.9910 - loss: 0.0283
 Test Loss: 0.02833825722336769
 Test Accuracy: 0.9909999966621399

```
predictions = model.predict(x_test)
```

```
y_pred = np.argmax(predictions, axis=1)
```

```
print("Predictions generated successfully!")
print("First 20 predictions:")
print(y_pred[:20])
```

313/313 ————— 2s 5ms/step
 Predictions generated successfully!
 First 20 predictions:
 [7 2 1 0 4 1 4 9 5 9 0 6 9 0 1 5 9 7 3 4]

```
precision = precision_score(
    y_test,
    y_pred,
    average="weighted"
)
```

```
recall = recall_score(
    y_test,
    y_pred,
    average="weighted"
)
```

```
f1 = f1_score(
    y_test,
    y_pred,
    average="weighted"
)
```

```
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
```

Precision: 0.9910309082123525
 Recall: 0.991
 F1-Score: 0.9909949240838032

```
print(
    classification_report(
```

```

classification_report(
    y_test,
    y_pred
)
)

```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	980
1	0.98	1.00	0.99	1135
2	0.99	0.99	0.99	1032
3	0.99	0.99	0.99	1010
4	0.99	1.00	0.99	982
5	0.98	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.98	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

```

cm = confusion_matrix(
    y_test,
    y_pred
)

plt.figure(figsize=(9, 7))

plt.imshow(cm)

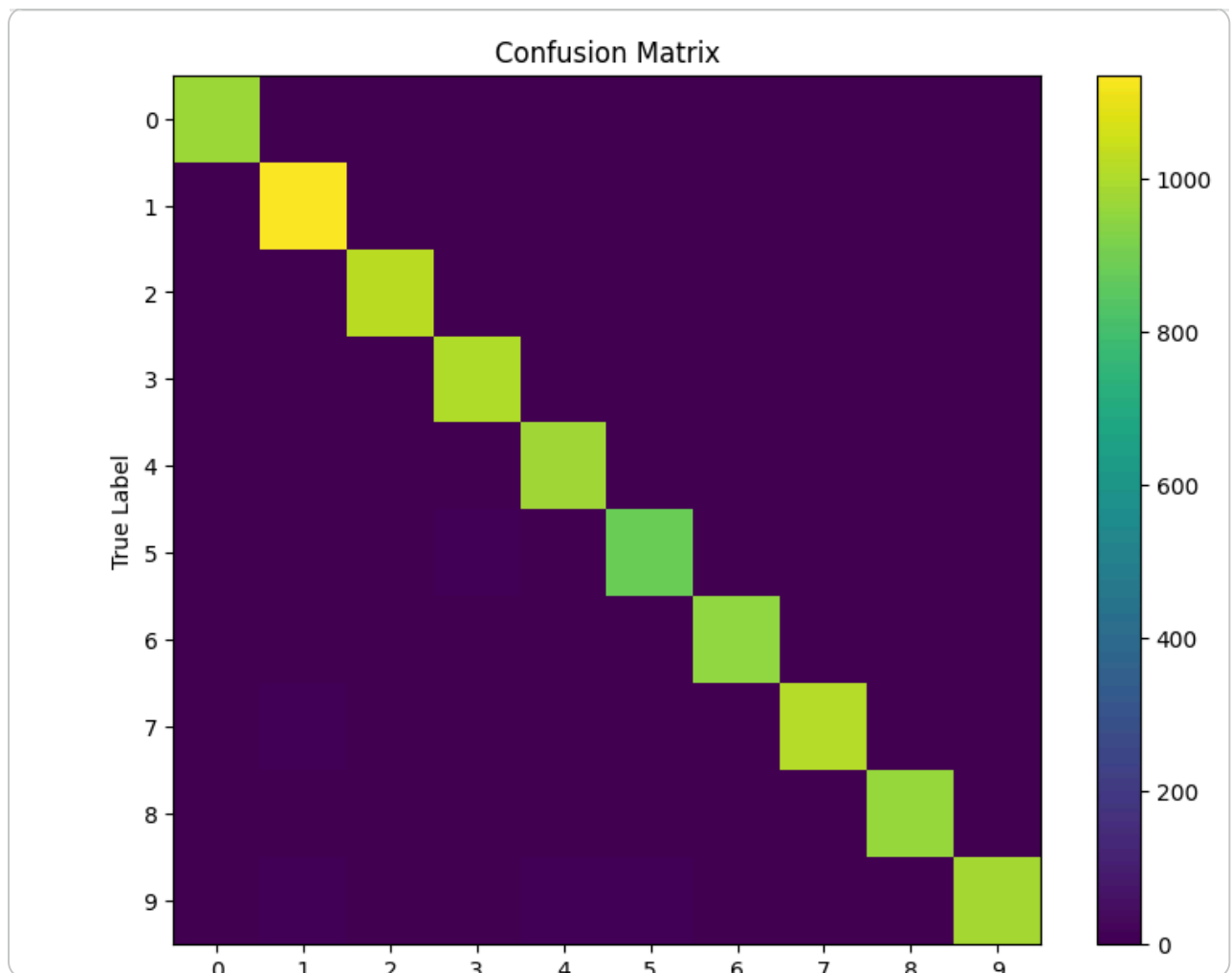
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.colorbar()

plt.xticks(range(10))
plt.yticks(range(10))

plt.show()

```



```
plt.figure(figsize=(12, 8))

for i in range(20):

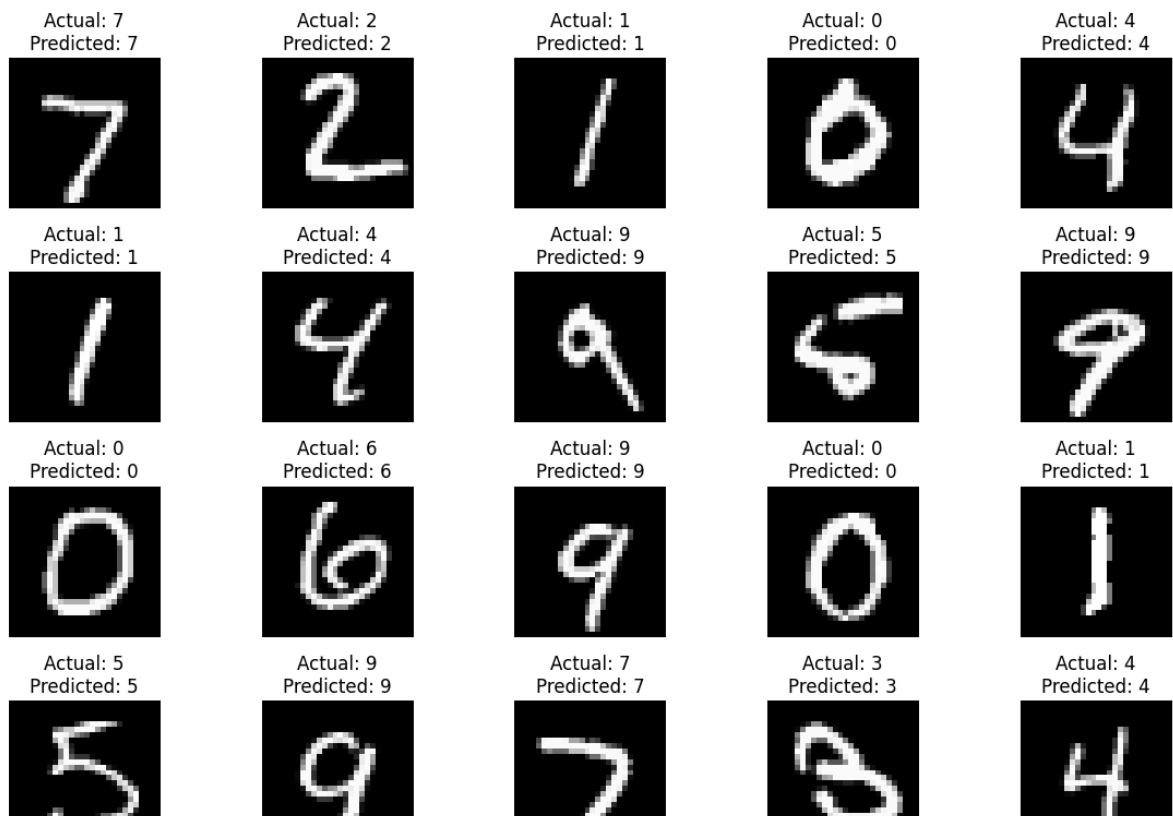
    plt.subplot(4, 5, i + 1)

    plt.imshow(
        x_test[i].reshape(28, 28),
        cmap="gray"
    )

    plt.title(
        f"Actual: {y_test[i]}\nPredicted: {y_pred[i]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
wrong_indices = np.where(
    y_pred != y_test
)[0]

print(
    "Total misclassified images:",
    len(wrong_indices)
)
```

Total misclassified images: 90

```
plt.figure(figsize=(12, 8))

for i in range(min(20, len(wrong_indices))):

    index = wrong_indices[i]

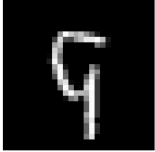
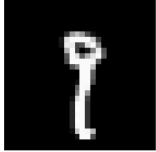
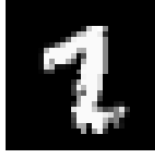
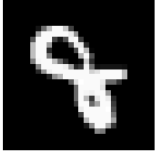
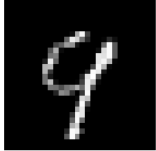
    plt.subplot(4, 5, i + 1)

    plt.imshow(
        x_test[index].reshape(28, 28),
        cmap="gray"
    )

    plt.title(
        f"Actual: {y_test[index]}\nPredicted: {y_pred[index]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```

Actual: 9 Predicted: 5 	Actual: 9 Predicted: 8 	Actual: 2 Predicted: 7 	Actual: 5 Predicted: 3 	Actual: 2 Predicted: 1 
Actual: 8 Predicted: 4 	Actual: 9 Predicted: 1 	Actual: 3 Predicted: 5 	Actual: 8 Predicted: 9 	Actual: 6 Predicted: 5 
Actual: 7 Predicted: 2 	Actual: 4 Predicted: 6 	Actual: 6 Predicted: 5 	Actual: 7 Predicted: 2 	Actual: 9 Predicted: 4 